

2026 KITS Hanoi

OJT Assignment Specification

SW Developer Track + IT Interpreter Track

0. Assignment Overview

Purpose	Team project at a practical, industry-level standard that allows company representatives to evaluate and make hiring decisions based on the final presentation
Participating Companies	Bzcom (SW Developer) / TechValley (SW Developer) / Genie Vina (SW Developer) / Both Companies (IT Interpreter)
Team Composition	4-5 members per team / Developer team and Interpreter team formed separately
Minimum Work Hours	At least 8 hours per team (outside of class hours)
Presentation Date	Graduation Ceremony
Presentation Format	PPT Presentation
Deliverables	PPT file + GitHub link (Developers) / PPT file + Interpretation log (Interpreters)

Developer Track — Tech Stack

Language	Java / Python
Architecture	MVC Pattern / MSA Concepts
API Design	REST API / Swagger (OpenAPI)
DB Design	ERD Design / JPA or In-Memory
Collaboration	Git Flow / Pull Requests / Commit Conventions
AI Utilization	LLM Integration / Prompt-based Development

PART 1. Bzcom — Developer Team Assignment

Company 1 Bzcom

Korea-Vietnam Web/App Development & Operations Company

Assignment: Customer Request Management System (CRM) REST API Design & Implementation

■ Background

Bzcom is an IT company that operates and maintains web services for 3 Korean client companies. Currently, bug reports, feature requests, and inquiries from clients come in through a mix of email, KakaoTalk, and phone calls. When team members change, the history disappears and missed requests keep recurring. Your team will design and implement a backend system to solve this problem. At the graduation ceremony, a Bizcom representative will evaluate your results directly and decide on hiring.

■ Step 1. ERD Design

Design an ERD diagram based on the table structure below. Clearly indicate relationships (PK/FK) between tables. Tools such as draw.io or dbdiagram.io are recommended.

[Tables to Design]

1. members
id, email, password, name,
role (ADMIN / DEVELOPER / CLIENT), createdAt
2. requests
id, title, description,
category (BUG / FEATURE / INQUIRY),
priority (HIGH / MEDIUM / LOW),
status (PENDING / IN_PROGRESS / DONE),
clientId → members, assignedDeveloperId → members,
createdAt, updatedAt
3. request_histories
id, requestId → requests, changedBy → members,
fromStatus, toStatus, changedAt, memo
4. alerts
id, requestId → requests, targetMemberId → members,
alertType (ASSIGNED / STATUS_CHANGED / HIGH_PRIORITY_REGISTERED),
message, isRead, createdAt

■ Step 2. REST API Design & Implementation

Swagger (OpenAPI) documentation is mandatory for all endpoints. Use a unified response format: { "status": 200, "message": "success", "data": {...} }

[Auth API]

POST	/api/auth/login	Login (JWT token issuance)
POST	/api/auth/logout	Logout
[Member API]		
POST	/api/members	Register member
GET	/api/members	Get all members (ADMIN only)
GET	/api/members/{id}	Get single member
[Request API]		
POST	/api/requests	Register request (CLIENT only)
GET	/api/requests	Get all requests (pagination / sorting / composite filters) ADMIN: all CLIENT: own only DEVELOPER: assigned only
GET	/api/requests/{id}	Get single request
PATCH	/api/requests/{id}/assign	Assign developer (ADMIN only / auto-assign logic)
PATCH	/api/requests/{id}/status	Update status (invalid transition prevention)
GET	/api/requests/{id}/history	Get status change history
GET	/api/requests/stats	Statistics (total, completion rate, by category, by developer)
[Alert API]		
GET	/api/alerts	Get own alert list
PATCH	/api/alerts/{id}/read	Mark alert as read
[LLM API – implement 1 of the following]		
POST	/api/requests/classify	Auto-classify category from description
POST	/api/requests/suggest-priority	Auto-suggest priority from description
GET	/api/requests/{id}/summary	Auto-generate 1-2 line summary

■ Step 3. Core Business Logic

1. JWT Authentication & Authorization
 - Issue Access Token upon login
 - Validate token on every API request
 - Role-based access control:
 - ADMIN: full management access
 - DEVELOPER: own assigned requests only
 - CLIENT: own registered requests only
2. Auto-Assignment Logic
 - Assign to the DEVELOPER with the lowest currentTaskCount
 - If tied, assign to the one who completed most recently
3. Status Transition Rules
 - Allowed: PENDING → IN_PROGRESS → DONE
 - Blocked: DONE → reverting to any other status
 - Blocked: PENDING → DONE (skipping IN_PROGRESS)
4. Automatic History Recording
 - Auto-record to request_histories on status change or developer assignment
5. Automatic Alert Generation
 - HIGH priority request registered → alert all ADMINS
 - Developer assigned → alert that developer
 - Status changed → alert the requesting CLIENT

■ Step 4. Git Flow

Branch Strategy

```

main           : final release
develop        : integration branch
feature/{name} : individual work branch

```

Rules

- Each member works on their own feature branch
- PR to develop with at least 1 team member review
- Commit message convention:
 - feat: new feature
 - fix: bug fix
 - docs: documentation
 - refactor: code refactoring
 - test: test code

Deliverable: GitHub link (commit history, branches, PRs included)

■ Step 5. PPT Presentation Structure (Graduation Ceremony)

#	Slide Title	Key Content
1	Cover	Team name, member intro, assignment title, company name
2	Problem Definition	Real problems Bizcom faces + scope of solution
3	ERD Design	Table structure + design rationale (why structured this way)
4	API Design Overview	Full endpoint list + RESTful design principles applied
5	Feature 1 — Auth	JWT flow + role-based access control structure
6	Feature 2 — Auto-Assign	Algorithm explanation + key code walkthrough
7	Feature 3 — History & Alerts	Status transition rules + auto-recording flow
8	LLM Feature	Chosen feature + Prompt design rationale + actual output
9	Swagger Demo	Screenshots of actual API calls
10	Git Flow	Branch structure + PR history + collaboration issues encountered
11	Individual Contributions	Each member's part + key learnings
12	Q&A	

■ Role Assignment Guide

Member	Assigned Part
Member A	Lead ERD design + Member & Auth API implementation
Member B	Request register, retrieve, filter, pagination API
Member C	Request assign, status change, history API
Member D	Alert API + common response format + exception handling + Swagger
Member E	LLM feature + Statistics API + Git management + PPT lead

※ ERD, common structure, and Git strategy must be agreed upon by the full team before implementation begins

■ Estimated Time

Phase	Content	Time
ERD + API structure alignment	Full team design discussion	60 min

Phase	Content	Time
Individual implementation	Each member builds assigned APIs	150 min
Integration + debugging	PR merge + conflict resolution	80 min
LLM feature completion	Prompt design + implementation	30 min
PPT production	Slide creation + content organization	80 min
Presentation rehearsal	Full run-through check	20 min
Total		approx. 8 hr 20 min

■ Hiring Evaluation Criteria

Evaluation Criteria	Hire: YES	Hire: RECONSIDERING
ERD Design	Relationships normalized, rationale clearly explainable	Field listing only, cannot explain relationships
REST API	RESTful principles followed, correct HTTP status codes	All handled via POST, verbs in URL
JWT Auth	Token flow understood, role-based control implemented	All APIs open with no authentication
Business Logic	Auto-assign, transitions, and history work correctly	Logic errors or unhandled exceptions
LLM Prompt	Purpose, context, and output format structured clearly	Generic "analyze this" level prompt
Git Flow	Branches, PRs, commit conventions actually followed	Direct push to main, meaningless commits
PPT Presentation	Design intent and technical decisions explained logically	Reads code only, no explanation of decisions

PART 2. TechValley — Developer Team Assignment

Company 2 TechValley

Cloud & SaaS Consulting Firm / Hanoi Vietnam Office

Assignment: Cloud Instance Monitoring System REST API Design & Implementation

■ Background

TechValley is an IT consulting firm providing Cloud & SaaS solutions to Vietnamese businesses. Currently, the status and costs of cloud instances for 10 client companies are managed manually in Excel, making real-time monitoring impossible and anomaly detection slow. Your team will design and implement an internal monitoring system to solve this problem. At the graduation ceremony, a TechValley representative will evaluate your results directly and decide on hiring.

■ Step 1. ERD Design

[Tables to Design]

1. members
id, email, password, name,
role (ADMIN / CLIENT_MANAGER), createdAt
2. clients
id, clientName,
contractPlan (BASIC / STANDARD / PREMIUM),
managerId → members, createdAt
3. instances
id, instanceName, region,
instanceType (SMALL / MEDIUM / LARGE),
status (RUNNING / STOPPED / ERROR),
cpuUsage, monthlyCost,
clientId → clients, launchedAt, updatedAt
4. alerts
id, instanceId → instances,
alertType (CPU_HIGH / ERROR_DETECTED / LONG_STOPPED),
message, isResolved, detectedAt, resolvedAt
5. cost_snapshots
id, clientId → clients,
snapshotMonth, totalCost, instanceCount, createdAt

■ Step 2. REST API Design & Implementation

```
[ Auth API ]
POST    /api/auth/login      Login (JWT token issuance)

[ Instance API ]
POST    /api/instances       Register instance
```

```

GET      /api/instances           Get all (pagination / filter / sort)
GET      /api/instances/{id}      Get single instance
PATCH   /api/instances/{id}/status Update status (RUNNING deletion blocked)
DELETE   /api/instances/{id}      Delete instance

[ Monitoring API ]
GET      /api/monitor/warnings    CPU ≥80% list + auto-record Alert
GET      /api/monitor/errors      ERROR status list + auto-record critical Alert
GET      /api/monitor/long-stopped STOPPED for 48+ hours list
GET      /api/monitor/report      Full status report
                                           (count by status / warning count / total cost / unresolved
Alerts)

[ Alert API ]
GET      /api/alerts              Alert history (date / type / resolved filter)
PATCH   /api/alerts/{id}/resolve Mark Alert as resolved

[ Client API ]
POST     /api/clients             Register client
GET      /api/clients             Get all clients
GET      /api/clients/{id}/instances Get instances by client
GET      /api/clients/{id}/cost    Monthly cost total by client
GET      /api/clients/{id}/cost-forecast Next month cost forecast
GET      /api/clients/{id}/sla     SLA uptime calculation
                                           PREMIUM 99.9% / STANDARD 99% / BASIC 95%

[ LLM API – implement 1 of the following ]
GET      /api/instances/{id}/diagnosis Auto-generate cause & action for ERROR instance
GET      /api/monitor/report/summary Executive summary of full report
GET      /api/clients/{id}/cost-optimization Cost reduction recommendations

```

■ Step 3. Core Business Logic

1. JWT Authentication & Authorization
 - ADMIN: full management of all clients and instances
 - CLIENT_MANAGER: view and manage assigned clients only
2. Automatic Alert Recording
 - On /api/monitor/warnings call: auto-record Alert for CPU ≥80% instances (skip if unresolved Alert already exists for that instance)
 - On /api/monitor/errors call: auto-record critical Alert for ERROR instances
3. Cost Forecast Logic
 - Unit pricing: SMALL \$50/mo MEDIUM \$120/mo LARGE \$250/mo
 - Calculate next month forecast based on current RUNNING instances
4. SLA Uptime Calculation
 - Calculate ratio of RUNNING time vs total hours in the month
 - Flag SLA violation if below contractPlan threshold
5. Instance Deletion Rules
 - RUNNING → deletion blocked (ActiveInstanceException)
 - STOPPED / ERROR → deletion allowed

■ Step 4. Git Flow

Same Git Flow rules as the Bizcom assignment. Submit GitHub link (commit history, branches, PRs included).

■ Step 5. PPT Presentation Structure (Graduation Ceremony)

#	Slide Title	Key Content
1	Cover	Team name, member intro, assignment title, company name
2	Problem Definition	Real problems TechValley faces + scope of solution
3	ERD Design	Table structure + rationale (why Alert & cost_snapshots tables added)
4	API Design Overview	Full endpoint list + RESTful design principles
5	Feature 1 — Auth	JWT flow + role-based access control
6	Feature 2 — Monitoring & Alerts	Warning detection + auto-recording + duplicate prevention
7	Feature 3 — Cost & SLA	Cost forecast logic + SLA evaluation structure
8	LLM Feature	Chosen feature + Prompt design rationale + output
9	Swagger Demo	Screenshots of actual API calls
10	Git Flow	Branch structure + PRs + collaboration issues
11	Individual Contributions	Each member's part + key learnings
12	Q&A	

■ Role Assignment Guide

Member	Assigned Part
Member A	Lead ERD design + Client & Auth API implementation
Member B	Instance register, retrieve, delete, filter, pagination API
Member C	Instance status change + full Monitoring API
Member D	Alert API + cost forecast + SLA calculation
Member E	LLM feature + Swagger + Git management + PPT lead

■ Estimated Time

Phase	Content	Time
ERD + API structure alignment	Full team design discussion	60 min
Individual implementation	Each member builds assigned APIs	160 min
Integration + debugging	PR merge + conflict resolution	80 min
LLM feature completion	Prompt design + implementation	30 min
PPT production	Slide creation + content organization	80 min
Presentation rehearsal	Full run-through check	20 min
Total		approx. 8 hr 30 min

■ Hiring Evaluation Criteria

Evaluation Criteria	Hire: YES	Hire: RECONSIDERING
ERD Design	Alert & cost_snapshot relationships designed logically	Simple field listing, relationships missing
Monitoring Logic	Alert dedup, auto-recording work correctly	Simple query only, no auto-recording
Cost & SLA	Calculations accurate, pricing basis explainable	Hardcoded dummy values returned
JWT Auth	Role-based access control actually works	All APIs fully public, no authentication
LLM Prompt	Domain context reflected in structured design	Generic question-level prompt

Evaluation Criteria	Hire: YES	Hire: RECONSIDERING
PPT Presentation	Design intent and technical decisions explained logically	Feature listing only

PART 3. Genie Vina — Developer Team Assignment

Company 3 Genie Vina

Vietnam IT/SW Vendor Intelligence Context for Korean Companies

Assignment: Vietnam IT Vendor Intelligence Mini Pipeline REST API Design & Implementation

■ Background

Korean companies reviewing potential IT/SW development partners in Vietnam often need to compare vendor information collected from public websites, company introductions, articles, LinkedIn, and directory sites. However, the information is scattered and difficult to compare in a structured way.

Your team will design and implement a backend-oriented Demo MVP that structures Vietnam IT/SW vendor information and allows users to register, retrieve, search, filter, classify, and summarize vendor data. At the graduation ceremony, a company representative will evaluate the result based on technical implementation, data structure, classification logic, collaboration, and presentation quality.

■ Step 1. ERD Design

Design an ERD diagram based on the table structure below. Clearly indicate relationships (PK/FK) between tables. Tools such as draw.io or dbdiagram.io are recommended.

[Tables to Design]

1. vendors
id, vendorCode, companyName, website, location,
serviceType (OUTSOURCING / SI / PRODUCT / CONSULTING / SPECIALIZED_TECH),
techStack, industryExperience, languageCapability,
companySize, classification, note,
createdAt, updatedAt
2. vendor_sources
id, vendorId -> vendors,
sourceType (PUBLIC_WEBSITE / DIRECTORY / LINKEDIN / ARTICLE / DEMO_DATA),
sourceUrl, sourceTitle, checkedAt, memo
3. classification_rules
id, classificationName,
description, judgmentCriteria, createdAt
4. classification_histories
id, vendorId -> vendors, changedBy -> members,
previousClassification, newClassification,
changedAt, reason
5. vendor_summaries
id, vendorId -> vendors,
summaryType (PROFILE_SUMMARY / LLM_SUMMARY / MANUAL_NOTE),
content, createdBy -> members, createdAt
6. members
id, email, password, name,
role (ADMIN / DEVELOPER / REVIEWER), createdAt

[Standard Classification Criteria]

- Outsourcing Vendor : software outsourcing or project-based development vendor
- SI Company : enterprise system implementation, integration, and maintenance
- Product Company : owns SaaS, platform, solution, or software product
- Consulting / IT Service : IT consulting, operation, project management, or advisory
- Specialized Tech Vendor : AI, Cloud, Data, Cybersecurity, Blockchain, etc.

[Minimum Vendor Data Fields]

vendorCode, companyName, website, location, serviceType, techStack,
industryExperience, languageCapability, companySize, sourceUrl,
classification, note

■ Step 2. REST API Design & Implementation

Swagger (OpenAPI) documentation is mandatory for all endpoints. Use a unified response format: { "status": 200, "message": "success", "data": {...} }

[Auth API]

POST /api/auth/login Login (JWT token issuance)
POST /api/auth/logout Logout

[Vendor API]

POST /api/vendors Register vendor
GET /api/vendors Get all vendors
(pagination / sorting / keyword search /
composite filters)
GET /api/vendors/{id} Get single vendor detail
PATCH /api/vendors/{id} Update vendor information
DELETE /api/vendors/{id} Delete vendor (admin only or soft delete)

[Source API]

POST /api/vendors/{id}/sources Add public source or demo-data note
GET /api/vendors/{id}/sources Get source list for a vendor
PATCH /api/vendors/{id}/sources/{sourceId} Update source information

[Classification API]

GET /api/classification-rules Get classification criteria
PATCH /api/vendors/{id}/classification Update vendor classification
GET /api/vendors/{id}/classification-history Get classification change history

[Statistics API]

GET /api/vendors/stats Statistics
(total, by classification, by location,
by service type)

[LLM API – Required: implement at least 1 of the following]

POST /api/vendors/classify Auto-suggest vendor classification
GET /api/vendors/{id}/summary Auto-generate 1-2 line vendor profile
summary
POST /api/vendors/extract-fields Extract structured fields from company
description

■ Step 3. Core Business Logic

1. JWT Authentication & Authorization

- Issue Access Token upon login
- Validate token on every protected API request
- Role-based access control:
 - ADMIN: full management access
 - DEVELOPER: register, update, search, classify, and summarize vendors

REVIEWER: read-only access to vendor data and classification results

2. Vendor Data Validation

- companyName, location, serviceType, techStack, sourceUrl/source note, and classification must be checked before saving
- website and sourceUrl may be optional only for educational demo data
- If source is unavailable, note must include "source unverified" or "demo data"

3. Source Control Rule

- Each vendor must include a public source URL or a clear demo-data note
- Non-public company information, private documents, and personal data are not allowed

4. Classification Logic

- Classification must be based on visible evidence such as service description, technology stack, industry experience, and company profile
- The result must not be presented as a real partner recommendation or ranking

5. Classification History Recording

- When classification is changed, automatically record previousClassification, newClassification, changedBy, changedAt, and reason

6. Search, Filter, and Statistics Logic

- Support keyword search by companyName, techStack, serviceType, or industryExperience
- Support filters by classification, location, technology stack, and language capability
- Provide at least one simple summary such as count by classification or location

7. LLM Utilization Rule

- LLM output is a reference only
- The final classification must be reviewed by the team and explained with evidence
- Prompt must include purpose, input fields, classification criteria, and output format

8. Scope Control

- The project is an educational Demo MVP
- It is not real vendor selection, due diligence, credit evaluation, sales DB construction, or partner recommendation

■ Step 4. Git Flow

Branch Strategy

```
main           : final release
develop        : integration branch
feature/{name} : individual work branch
```

Rules

- Each member works on their own feature branch
- PR to develop with at least 1 team member review
- Commit message convention:
 - feat: new feature
 - fix: bug fix
 - docs: documentation
 - refactor: code refactoring
 - test: test code

Deliverable: GitHub link (commit history, branches, PRs included)

■ Step 5. PPT Presentation Structure (Graduation Ceremony)

#	Slide Title	Key Content
1	Cover	Team name, member intro, assignment title, company name
2	Problem Definition	Real business problem Genie Vina faces + scope of solution
3	ERD Design	Table structure + design rationale for vendor, source, classification, and summary tables
4	API Design Overview	Full endpoint list + RESTful design principles applied
5	Feature 1 — Vendor Data Management	Register, retrieve, update, search, and filter logic
6	Feature 2 — Classification Logic	Classification criteria + history recording + evidence-based explanation
7	Feature 3 — Source Control & Statistics	Source URL/demo-data note rule + basic statistics
8	LLM Feature	Chosen feature + prompt design rationale + actual output
9	Swagger Demo	Screenshots of actual API calls
10	Git Flow	Branch structure + PR history + collaboration issues encountered
11	Individual Contributions	Each member's part + key learnings
12	Q&A	

■ Role Assignment Guide

Member	Assigned Part
Member A	Lead ERD design + Vendor registration API implementation
Member B	Vendor retrieve, search, filter, pagination, and detail API
Member C	Classification API + classification history + statistics API
Member D	Source API + common response format + exception handling + Swagger
Member E	LLM feature + Git management + PPT lead

※ ERD, API structure, classification criteria, and Git strategy must be agreed upon by the full team before implementation begins

■ Estimated Time

Phase	Content	Time
ERD + API structure alignment	Full team design discussion	60 min
Individual implementation	Each member builds assigned APIs	160 min
Integration + debugging	PR merge + conflict resolution	80 min
LLM feature completion	Prompt design + implementation	30 min
PPT production	Slide creation + content organization	80 min
Presentation rehearsal	Full run-through check	20 min
Total		approx. 8 hr 30 min

■ Hiring Evaluation Criteria

Evaluation Criteria	Hire: YES	Hire: RECONSIDERING
ERD Design	Vendor, source, classification, and history relationships are designed logically	Simple field listing only, relationships missing
REST API	RESTful principles followed, Swagger documented, correct response format used	All handled as ad-hoc functions, no clear API design
Business Logic	Validation, source control, classification, search/filter, and statistics work correctly	Logic errors, arbitrary classification, or unhandled exceptions

Evaluation Criteria	Hire: YES	Hire: RECONSIDERING
LLM Prompt	Purpose, context, input fields, output format, and verification method are structured clearly	Generic prompt with no business context or verification
Git Flow	Branches, PRs, and commit conventions actually followed	Direct push to main or meaningless commits
PPT Presentation	Design intent, classification rationale, limitations, and technical decisions explained logically	Feature listing only, no explanation of decisions

PART 4. Interpreter Team Assignment — Common to Both Companies

Common IT Company Field Interpretation Project

Select one of Bzcom, TechValley, or Genie Vina as the assigned company

Assignment: IT Company Field Interpretation Project

■ Background

You are assigned to one of the following company cases as an interpreter: Bzcom, TechValley, or Genie Vina. You will interpret a variety of Korean-Vietnamese communication situations that arise in real work environments. At the graduation ceremony, you will present your interpretation skills in front of company representatives, who will evaluate and decide on hiring.

■ Step 1. Individual Pre-Assignment

Before the team assignment begins, each member completes the following two items individually.

Item	Details
IT Glossary	Minimum 30 entries / individual submission Korean Vietnamese English Example sentence in context Category classification required: Development / Infrastructure / Project Management / Business
Company Research Report	1 page (A4) / individual submission ① Main business areas ② What the company does in Vietnam ③ Top 3 core competencies required for an interpreter at this company ④ 3 anticipated interpretation scenarios

■ Step 2. Team Role Assignment

Role	Responsibilities
Lead Interpreter 1 (1 person)	Scenes 1–4: perform live interpretation
Lead Interpreter 2 (1 person)	Scenes 4–7: perform live interpretation
Terminology Support (1 person)	Provide real-time written support for IT terms during interpretation
Quality Monitor & Presentation Lead (1 person)	Check for omissions and mistranslations, compile post-session feedback. PPT production + presentation facilitation + interpretation log compilation

※ Lead interpreters may rotate roles after rehearsal / team may adjust assignments as needed

■ Step 3. Interpretation Scenarios (7 Scenes)

Teams perform based on scripts prepared by the instructor for each company.

Bzcom Scenarios (7 Scenes)

Scene 1. New Member Onboarding — Korean PM explains project structure and workflow to new Vietnamese developer

Scene 2. Bug Report & Fix Instruction — Client reports bug → PM instructs developer on root cause investigation and fix

Scene 3. Feature Requirement Handoff — Korean client requests new feature / Development team explains technical constraints

Scene 4. Code Review Meeting — Senior developer reviews junior code / explains improvements and takes questions

Scene 5. Schedule Negotiation — PM and dev team coordinate deadline / discuss resource shortage situation

Scene 6. Pre-Deployment Final Check — Review deployment checklist / staging-to-production deployment approval process

Scene 7. Client Complaint Handling — Client protests service outage / PM explains situation and provides response plan

TechValley Scenarios (7 Scenes)

Scene 1. Cloud Migration Proposal — Consultant explains to Vietnamese client executives why cloud migration is necessary

Scene 2. Technical Requirements Gathering — Interview with client IT manager about current infrastructure situation

Scene 3. Service Outage Response — Client server failure occurs / consultant explains root cause and remediation

Scene 4. Migration Progress Report — Weekly meeting progress update / explains schedule delay and negotiates adjustment

Scene 5. Cost Negotiation — Explains cloud cost structure to client CFO / discusses cost-reduction options

Scene 6. Staff Training — Consultant trains client staff on using the new system

Scene 7. Contract Renewal Negotiation — Re-negotiates scope and pricing as contract approaches expiration

Genie Vina Scenarios (7 Scenes)

Scene 1. Company Briefing — Introduce Genie Vina's business areas, current products, and team structure to a Korean company representative

Scene 2. Project Requirement Discussion — Interpret a functional requirements meeting between a Korean PM and a Vietnamese development team, ensuring scope and priorities are clearly communicated

Scene 3. API and Technical Spec Review — Interpret a technical review session covering REST API design, endpoint naming conventions, and response format decisions between Korean and Vietnamese developers

Scene 4. LLM and AI Feature Explanation — Explain LLM-based vendor classification and summarization features to a non-technical Korean stakeholder, clarifying how AI output is used and verified

Scene 5. Data Classification Discussion — Facilitate a meeting to define vendor classification criteria (Outsourcing / SI / Product / Consulting / Specialized Tech), ensuring both sides agree on judgment standards

Scene 6. QA and Bug Report — Interpret a QA session where the Vietnamese team reports bugs and test results to a Korean reviewer, accurately conveying technical defect descriptions and reproduction steps

Scene 7. Final Demo Preparation Meeting — Interpret the final pre-presentation alignment meeting, summarizing unresolved technical terms, communication risks, and key messages to confirm before the graduation ceremony demo

■ Step 4. Rehearsal + Team Feedback

Phase	Content	Notes
1st Rehearsal	Lead interpreters split scenes and perform all 7	Terminology support & quality monitor active simultaneously
Team Internal Feedback	Quality monitor shares findings / team resolves difficult expressions / updates glossary	approx. 30 min
2nd Rehearsal	Re-perform with 1st feedback incorporated / quality monitor confirms improvements	Role rotation possible

■ Step 5. PPT Presentation Structure (Graduation Ceremony)

#	Slide Title	Key Content
1	Cover	Team name, member intro, assigned company name
2	Company Research Summary	Business areas + top 3 competencies required for interpreters
3	Team Role Assignment	Each member's role + rationale for division
4	IT Glossary Highlights	Key terms by category (10 entries) + real-use example sentences
5–9	Scene Interpretation Demo	Select 5 most challenging scenes / scene description + live demo (1–2 min) + difficult expression solutions
10	Rehearsal Improvement Cases	1st vs 2nd rehearsal improvements + quality monitor feedback summary
11	Individual Takeaways	Most challenging situation + how it was solved + growth areas
12	Q&A	

■ Estimated Time

Phase	Content	Time
Individual pre-assignment	Glossary + company research (individual)	90 min
Role assignment alignment	Full team discussion	20 min
1st rehearsal	Full 7-scene run	60 min
Team feedback session	Correct mistranslations and omissions	30 min
2nd rehearsal	Re-run with improvements applied	50 min
PPT production	Slide creation + demo preparation	80 min
Presentation rehearsal	Full run-through check	20 min
Total		approx. 8 hr 10 min

■ Hiring Evaluation Criteria

Evaluation Criteria	Hire: YES	Hire: RECONSIDERING
Terminology Preparation	30+ entries with real-use example sentences	Words listed only, no examples
Interpretation Accuracy	IT terms, numbers, conditions delivered accurately	Key content omitted or mistranslated
Improvement Process	Clear improvement from 1st to 2nd rehearsal confirmed	Same mistakes repeated in both sessions
Team Collaboration	Roles clearly divided, members support each other	Only lead interpreters work, others observe
Presentation Quality	Difficult expressions and solutions explained logically	Demo only, no explanation of process
Company Understanding	Tone and expressions match the company's business context	Context-free literal translation throughout